SETTING UP KUBERNETES ON AWS (EKS)

CREATION OF EKS CLUSTER ON AWS USING GUI

1.Create A Cluster

1. Custom configuration
2. eks auto mode ---> disabled
3. name ----> eks
4. Cluster IAM role ----> create a recommended role
    1. AmazonEKSClusterPolicy
    2. AmazonEKSComputePolicy
    3. AmazonEKSNetworkingPolicy
    4. AmazonEKSLoadBalancingPolicy
5. version --> 1.35
6. standard support
7. choose_subnets ---> All Subnets
8. cluster endpoint ---> public & private
9. create a cluster

2.Create A Worker Node

1. Click on cluster ----> compute ----> node group ----> add
2. Node IAM role -----> create recommended role
    1. AmazonEKSWorkerNodePolicy
    2. AmazonEKS_CNI_Policy
    3. AmazonEC2ContainerRegistryReadOnly
3. instance type ---> c7i-flex. large
4. storage ----> 20 gb
5. desired-2 minimum-2 maximum-2
6. node-group-update-config ------> 1
7. subnets ---> all subnets
8. create

3.Get Access On Local Terminal (Ubuntu)

1. aws configure
   access_key:
   secret_key:
   default region: us-east-1

2. aws eks update-kubeconfig --name eks --region us-east-1
3. kubectl get nodes

4.Create 1st Pod

1. kubectl run mypod --image=nginx --port=80
2. kubectl get pods

CREATION OF EKS CLUSTER ON AWS USING CLI

1. create a EC2 instance (amazon linux)
2. Create a role(admin) and attach to a instance
3. Access the instance

1.Install Kubectl

1. curl -O https://s3.us-west-2.amazonaws.com/amazon-eks/1.35.3/2026-04-08/bin/linux/amd64/kubectl
2. chmod +x ./kubectl
3. mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export PATH=$HOME/bin:$PATH
4. echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
5. kubectl version --client

2.Install eksctl

{ https://docs.aws.amazon.com/eks/latest/eksctl/installation.html }

3.Create A Cluster

1. eksctl create cluster --name eks --region us-east-1 --version 1.35 --nodegroup-name workers --node-type c7i-flex.large --nodes 2 --managed

4.Connet To EKS & Check

1. aws eks update-kubeconfig --region us-east-1 --name eks
2. kubectl get nodes

6.To Delete A Cluster

1. eksctl delete cluster --name eks --region us-east-1

INSTALLATION OF MINIKUBE

1.Create A Instance

1. name ------> minikube
2. type --------> m7i-flex.large
3. sg ----------> ssh / http / https
4. create a instance

2.Install Docker

{ https://docs.docker.com/engine/install/ubuntu/ }

3.Install Kubectl

1. sudo snap install kubectl --classic

4.Install Minikube

1. curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
2. sudo install minikube-linux-amd64 /usr/local/bin/minikube
3. minikube start --force
4. minikube version

INSTALLING THE KIND-CLUSTER

1.Create A Instance

1. name --------> kind-cluster
2. type ----------> m7i-flex. large
3. sg -------------> http/https/ssh
4. create a instance

2.Install Docker

1. apt update
2. apt install docker -y
3. systemctl start docker
4. systemctl status docker

3.Install Kubectl

1. curl -O https://s3.us-west-2.amazonaws.com/amazon-eks/1.35.3/2026-04-08/bin/linux/amd64/kubectl
2. chmod +x ./kubectl

3. mkdir -p $HOME/bin && cp ./kubectl $HOME/bin/kubectl && export PATH=$HOME/bin:$PATH
4. echo 'export PATH=$HOME/bin:$PATH' >> ~/.bashrc
5. kubectl version --client

4.Installing Kind-Cluster

1. vim install_kind.sh

```
[ $(uname -m) = x86_64 ] && curl -Lo ./kind
https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
chmod +x ./kind
sudo cp ./kind /usr/local/bin/kind
rm -rf kind
```

2. chmod +x install_kind.sh
3. bash kind.sh
4. vim config.yaml

```
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
nodes:
- role: control-plane
  image: kindest/node:v1.30.0
- role: worker
  image: kindest/node:v1.30.0
- role: worker
  image: kindest/node:v1.30.0
```

5. kind create cluster --config config.yml --name my-cluster
6. kubectl get nodes
7. kubectl get pods

INSTALLATION OF KUBERNETES ON KUBEADM

1.create a instance
    1. master_node
    2. worker_node

ON BOTH MASTER AND WORKER NODE

1.sudo su
2.sudo apt-get update

1.vim script.sh

```
#!/bin/bash
# disable swap
sudo swapoff -a
# Create the .conf file to load the modules at bootup
cat <<EOF | sudo tee /etc/modules-load.d/k8s.conf
overlay
br_netfilter
EOF
sudo modprobe overlay
sudo modprobe br_netfilter
# sysctl params required by setup, params persist across reboots
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
# Apply sysctl params without reboot
sudo sysctl --system
## Install CRIO Runtime
sudo apt-get update -y
sudo apt-get install -y software-properties-common curl apt-transport-https ca-certificates gpg
sudo curl -fsSL https://pkgs.k8s.io/addons:/cri-o:/prerelease:/main/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/cri-o-apt-keyring.gpg
echo "deb [signed-by=/etc/apt/keyrings/cri-o-apt-keyring.gpg] https://pkgs.k8s.io/addons:/cri-o:/prerelease:/main/deb/ /" | sudo tee /etc/apt/sources.list.d/cri-o.list
sudo apt-get update -y
sudo apt-get install -y cri-o
sudo systemctl daemon-reload
sudo systemctl enable crio --now
```

sudo systemctl start crio.service
echo "CRI runtime installed successfully"
Add Kubernetes APT repository and install required packages
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.29/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg
echo 'deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.29/deb/ /' | sudo tee /etc/apt/sources.list.d/kubernetes.list
sudo apt-get update -y
sudo apt-get install -y kubelet="1.29.0-*" kubectl="1.29.0-*" kubeadm="1.29.0-*"
sudo apt-get update -y
sudo apt-get install -y jq
sudo systemctl enable --now kubelet
sudo systemctl start kubelet

2.bash script.sh

ON MASTER_NODE ONLY

1.initialize cluster
kubeadm init --pod-network-cidr=192.168.0.0/16 --cri-socket=unix:///var/run/crio/crio.sock
{ COPY the kubeadm join command shown at the end}

2.Configure Kubectl
mkdir -p $HOME/.kube
cp /etc/kubernetes/admin.conf $HOME/.kube/config
chown $(id -u):$(id -g) $HOME/.kube/config

2.Install Calico (NETWORK)
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.0/manifests/calico.yaml

ON WORKER_NODE ONLY

1. run kubeadm join command on worker-node and at last run --cri-socket unix:///var/run/crio/crio.sock --v=5
2. add the port in sg that u get in the terminal on master-node

ON MASTER_NODE

1. kubectl get nodes

HIGHER LEVEL OBJECTIVES IN KUBERNETES

2. Pod
3. ReplicaSet
4. Replication Controller
5. Deployment
6. StatefulSet
7. DaemonSet
8. Service
9. Ingress
10. Namespace
11. ConfigMap
12. Secret
13. PersistentVolume (PV)
14. PersistentVolumeClaim (PVC)
15. Horizontal Pod Autoscaler (HPA)

Difference Between Main Container / Init Container / Side Car Containers

1. Main Container

- Primary application container
- Runs the main application
- Pod mainly exists because of this container

2. Sidecar Container

- Side-car containers are those containers that run alongside with the main container
- We called it as the helper container
- Shares same network and storage with main container
- Use-cases:
    1. Logging
    2. Monitoring
    3. Proxy
    4. Data sync

3. Init Container

- Init containers are those containers that run before the app container in a pod (run before main container starts)
- Executes setup tasks
- If the init container fails the main container should not be start.
- Use-cases:
    1. Download configuration
    2. Setup files/directories
    3. Check dependencies

Run Our 1st Pod

1. kubectl run mypod --image=nginx --port=80
2. kubectl get pods
3. kubectl get pods -o wide
